REPORT · 2026年10月

图集跨件复用与导出加速

中小型文物三维建模自动化系统 v1.1.6-r10 技术分析

文保数字化研发组
2026年10月

目 录

一、结论摘要 3

二、为什么可以复用：判据链与反例 5

2.1 事实链（本次 5 件的具体情况） 5

2.2 反例（必须判为不同、必须重算） 5

三、实现 7

3.1 模块与职责 7

3.2 缓存布局与键 7

3.3 命中路径（M4 内部） 7

3.4 三条硬边界与自愈 8

3.5 M3 泊松不可复现：打包版端到端测出来的第三个问题 8

3.6 导出加速：Open3D 写 OBJ 的问题 9

四、实测数据 10

4.1 关闭复用（基线） 10

4.2 开启复用 10

4.3 画质不变的证据 10

4.4 与上一轮真机数据的关系 10

4.5 打包版端到端（2 件同源输入） 11

五、测试与验收 12

六、风险与未做 13

七、下一步 14

一、结论摘要

上一版（v1.1.5）把 S1 的重复解算按**照片内容指纹**收口后，纯照片场景 5 件的SfM 从 1801 s/件降到 0.2~0.3 s/件。代价是瓶颈整体前移：**M4（照片投影贴图）成为唯一的大头**——真机实测 83~85 s/件，其中 UV 展开 33 s、逐纹素多视角采样与融合 50 s。而这 5 件的 fused_dense.ply 与 capture_meta.json 是 S1 复用**逐字节复制**出来的：几何一样、位姿一样、照片一样，那么 UV 展开与逐纹素采样的结果**必然逐像素相同**，却被按件各算了一遍。

本批按「能证明相同的东西不必重算」的思路做了两件事：① 为 M4 增加**两级图集缓存**（UV 级 / 纹理级），命中即直接取图集、只重做导出；② 把复用后暴露出来的新大头——**导出**——换掉：Open3D 写一个带纹理 OBJ 要 5.34 s，改为手写后 0.21 s。

核心结论：① 5 件同照片集，M4 由 22.5~24.4 s/件 降到**第 2 件起 0.5~0.6 s/件**（整批 115 s → 29 s，单件省 97.7%）；② 5 件的贴图与着色掩膜 **SHA-1 完全相同**，且与关闭复用的那一轮**逐字节一致**——省的是时间，不是画质；③ 带纹理 OBJ 导出 5.34 s → **0.21 s**（25 倍），回读后几何与逐角 UV 与源网格一致；④ 打包版端到端又暴露并修掉一个更根本的问题：**M3 泊松重建默认多线程、逐位不可复现**（同一份点云三次三个哈希），它既破坏论文的「同输入同输出」，又让图集复用永不命中——改为单线程后恢复可复现，代价 0.84 s → 1.51 s；⑤ 全量测试 **705 项通过**（较 1.1.5 的 682 项 +23）；⑥ 复用三边界、损坏自愈、开关与三处留痕齐备，不会把 A 件的贴图按到 B 件上。

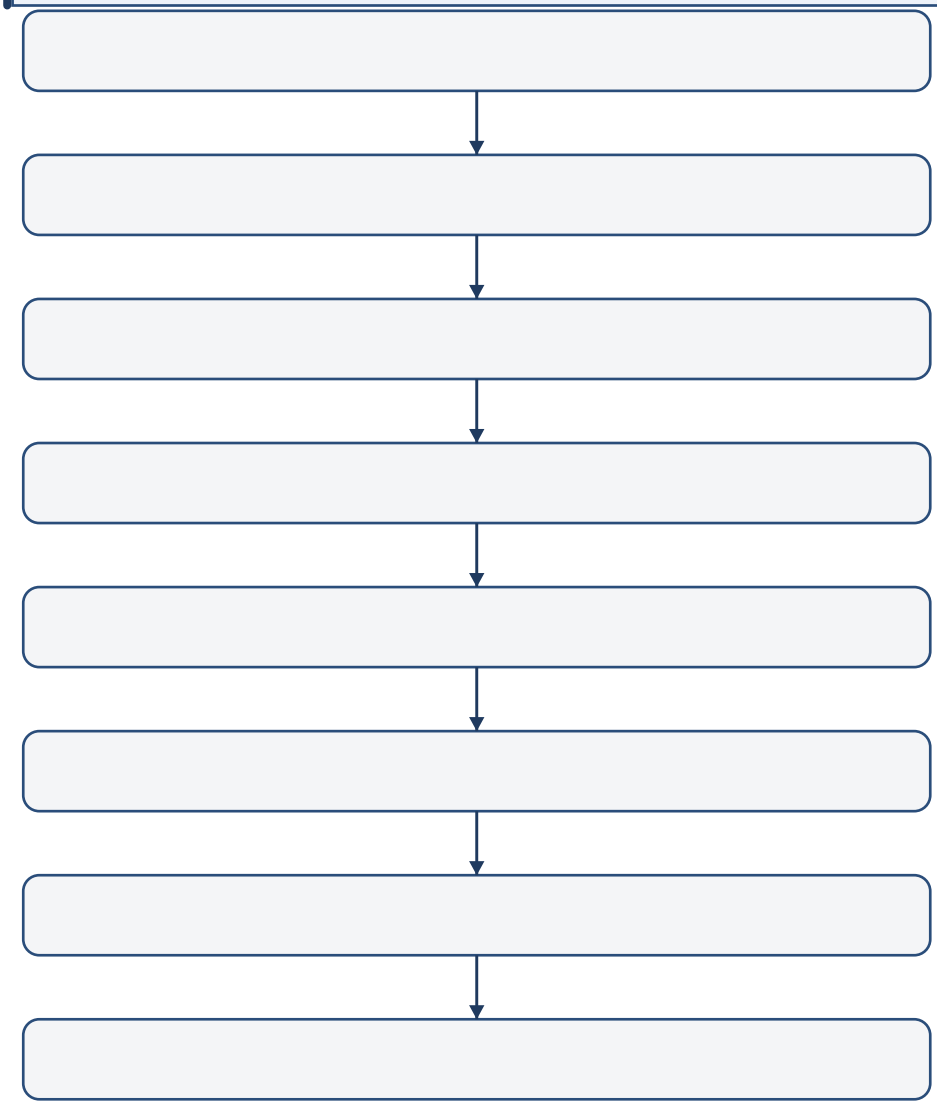


图 1 本批工作链路（瓶颈前移 -> 复用判据 -> 导出加速 -> 验收）

二、为什么可以复用：判据链与反例

复用必须先能**证明两份结果相同**，否则就是把不同器物的贴图混淆。此处把 M4 的两次重计算拆成两个独立的依赖集，分别建键：

重计算项	实测耗时	只依赖	可用的键	复用粒度
xatlas UV 展开	9.9~33 s	几何（顶点坐标 + 面索引）	几何指纹	几何相同的所有件（照片不同也能命中）
逐纹素多视角采样 + 泊松融合	11.9~50 s	几何 + 法线 + 相机位姿 + 照片像素 + 采样参数	五项指纹	几何/位姿/照片/参数全同的件
导出（obj/mtl/png/glb/ply）	0.4~6.1 s	网格 + 图集 + **件号**	不缓存	每件重做（文件名含件号，复制还要改 MTL 引用）

这个拆法的意义在于 **UV 级的命中条件比纹理级宽得多**：只要几何相同，即使照片完全不同（例如同一件器物换了一批补拍照片），UV 展开仍可复用；反之，纹理级命中必然连带 UV 相同，所以两级可以独立命中、独立留痕。

2.1 事实链（本次 5 件的具体情况）

环节	证据	结论
S1 复用	5 件 00_photo 各 30 张 DSC_*.JPG，名+大小+内容 SHA-1 均为 d58930a998ac470c	同一批照片被复制成 5 份目录
S1 产物	复制 10_dense/<件>/fused_dense.ply 与 90_meta/<件>_capture_meta.json	几何与位姿**逐字节相同**
M3 网格	由同一 ply 走同一套 Poisson 参数	网格相同
M4	同一网格 + 同一组位姿 + 同一批照片 + 同一组参数	UV 与纹理**必然同结果**

2.2 反例（必须判为不同、必须重算）

情形	UV 级	纹理级	理由
几何相同、照片不同	命中	未命中	UV 只由几何决定；纹理取决于像素
几何相同、位姿微调	命中	未命中	采样按相机投影，位姿变了颜色就变
几何不同（分离 1e-3 量级）	未命中	未命中	指纹按 float32 顶点精确比较
几何相同、仅法线符号翻转	命中	未命中	法线参与采样判据与补色
改了 texture_size / top_k / 平滑参数	命中	未命中	参数进 key
不同项目根（各自 99_runs）	未命中	未命中	缓存按项目根隔离，不跨项目串味

指纹的精度取 **float32**：同一网格经 ply 读写、变换链可能带 1e-10 级 double 末位噪声，量化到 float32 后仍逐位相同（已用单测锁定：坐标加 1e-10 不改变指纹）；而 1e-3 级的真实位移必然改变指纹。

顶点**顺序**参与哈希，因为 UV 的 `vmap` 按原顶点下标索引，顺序不同不能通用。

三、实现

3.1 模块与职责

文件	内容	说明
src/h3d/texture/atlas_cache.py（新增）	AtlasCache + 4 个指纹函数 + 原子写/损坏留档	缓存本体；与 S1 复用同一套「指纹 + 索引 + 三条边界」思路
src/h3d/texture/uv.py	unwrap_mesh(..., return_vmap=True)、rebuild_with_uv()、_write_obj_textured()	命中时用 vmap 重建展开网格；手写 OBJ 快路径
src/h3d/photo_texture.py	texture_mesh_from_photos() 内接入两级缓存	M4 与 P1 共用这个函数，故 P1 同样受益
src/h3d/core/config.py	photo_texture.reuse_atlas（默认 true）	可关
src/h3d/mesh/poisson.py / core/config.py	n_threads=1 默认 + mesh.poisson_threads	M3 恢复逐位可复现（图集复用的前提）
src/h3d/health/_project.py	新增 atlas_cache 检查项 + 同名照片提示补充	doctor 里可见缓存套数与体积

3.2 缓存布局与键

```
<项目根>/99_runs/atlas_cache/
├── uv/
│   ├── v1_<几何指纹 20 位>.npz      vmap / indices / uvs
│   └── v1_<几何指纹 20 位>.json     source_item / created / n_verts / n_faces
└── tex/
    ├── <纹理键 24 位>.png          贴图（uint8 RGB，与交付 PNG 同精度）
    ├── <纹理键 24 位>.mask.png      着色掩膜（M5 用它算填充率）
    └── <纹理键 24 位>.json         source_item + 首件的完整 metrics

UV 键    = sha1(geom-v1 | float32 顶点 | int32 面)
纹理键  = sha1(v<CACHE_VERSION> | 几何指纹 | 法线指纹 | 位姿指纹 |
           照片内容 SHA-1 | 采样参数指纹)
```

为什么要 CACHE_VERSION：几何/纹理语义一旦改动（换 UV 展开算法、改加权公式、改泊松实现），旧缓存会把「上一版算法」的结果伪装成当前算法的产出，且不报错。版本号进键名与目录名，升级即自然失效。

3.3 命中路径（M4 内部）

```
# photo_texture.texture_mesh_from_photos()（节选）
cache = AtlasCache(task_root, enabled=cfg.photo_texture.reuse_atlas)
geom_fp = mesh_fingerprint(mesh)              # 只含几何
hit = cache.get_uv(geom_fp)
if hit is not None:
    vmap, indices, uvs, meta = hit
    if 下标自洽:                              # 越界会被 Open3D 抛 C++ 异常
        new_mesh, uv, faces = rebuild_with_uv(mesh, vmap, indices, uvs)
        reuse["uv"] = "hit"; reuse["uv_from"] = meta["source_item"]
if hit is None:
    new_mesh, uv, faces, vmap = unwrap_mesh(mesh, return_vmap=True)
    cache.put_uv(geom_fp, vmap, faces, uv, item_id)

tex_key = cache.texture_key(geom_fp, mesh_fingerprint(new_mesh, with_normals=True),
                            pose_fingerprint(cameras, images),
```

```
                    photo_fingerprint(photo_dir),
                    params_fingerprint(cfg))
if (tex := cache.get_texture(tex_key)) is not None and 尺寸与配置一致:
    image, painted, m = tex                  # metrics 随图集一起复用
else:
    image, metrics, painted = project_photos_to_texture(...)
    image = poisson_blend(...)
    cache.put_texture(tex_key, image, painted, metrics, item_id)
```

注意两处细节：① 缓存命中也必须把**质量指标**一并取回（覆盖率、图集利用率、参与相机数、多视角一致性…），否则报表会把「复用件」显示成 0 覆盖率，等于给出一份假数据；② 纹理尺寸要与当前配置二次核对——键里虽然已有 `texture_size`，但缓存文件可能被外部改写，尺寸不符时按未命中重算而不是硬贴。

3.4 三条硬边界与自愈

边界	实现	为什么必须有
缺一不复用	纹理缓存要求 `.png` + `.mask.png` + `.json` 三件齐全；UV 要求 vmap/indices/uvs 形状自洽	只有一半的缓存会拼出畸形结果（无掩膜则 M5 填充率失真）
参数不一致不复用	texture_size / top_k / samples_per_tri / poisson_blend 全部进 key；照片内容走 SHA-1	改参数是常规操作，静默复用等于让参数失效
三处留痕	日志一行、manifest `atlas_reuse`、报表 metrics 全量	复用必须是**可审计**的，不能靠猜
损坏可恢复	读失败 → 改名 `.corrupt-<时间戳>` + WARNING + 按未命中重算	派生状态损坏只该损失时间，不该让项目跑不起来
体积有上限	按 mtime 只保留最近 24 套（1024² 约 3 MB/套）	批量任务不能把盘吃满

3.5 M3 泊松不可复现：打包版端到端测出来的第三个问题

上面两级缓存做完、源码级基准也全绿之后，我把 **打包版 exe** 拉到一个「两件完全相同的输入」的小项目上跑 `--batch` 做端到端验证——结果 **缓存一次也没命中**，而且两件的 `30_dense` 面数是 19830 与 19828。

逐阶段比哈希后定位：`10_clean` 与 `20_registered` 两件**逐字节相同**，分叉只发生在 M3。把 Open3D 的泊松单独拎出来跑三次，同一份点云得到顶点数与面数都一样（10244 / 20484）但坐标与面表哈希全不同的三个结果：

运行	顶点数	面数	顶点哈希	面表哈希
默认（多线程）第 1 次	10244	20484	80492b8f0d7b	d1d8bbc54714
默认（多线程）第 2 次	10244	20484	812b5fa5dfac	0bd1d3e208ab
默认（多线程）第 3 次	10244	20484	c9602497bb86	6fc88efbdaf0
`n_threads=1` 第 1 次	10244	20484	00989fb5799d	515caafe8efd
`n_threads=1` 第 2 次	10244	20484	00989fb5799d	515caafe8efd

这不是精度噪声：并行归约顺序不同会让线性方程组解在末位分叉，再被三角化放大成不同的面表。后果有两层：① 论文的「同输入同输出」原本就不成立——与本项目此前踩过的「COLMAP 多线程导致 database.db 不可复现」属同一类问题；② 图集复用按几何指纹判等，网格逐位不同就**永远不命中**，我这两级缓存在真实多件场景里会形同虚设。

> **修复**：poisson_reconstruct(..., n_threads=1) 作为默认，新增 mesh.poisson_threads 配置（默认 1）。代价实测 37k 点 / depth=8：0.84 s → **1.51 s**，远小于它解锁的 22 s/件。修复后打包版 2 件同源项目跑通：第 2 件日志出现「UV 图集复用 / 纹理图集复用（来自 demo_a）」，total=2 done=2 failed=0。

这条经验值得单独记一笔：**单元测试与源码级基准都通过，不等于打包版端到端可用。**缓存类优化的「命中率」只有在真实的多件批处理里才暴露得出来——我的基准是手工把同一份 30_dense 网格喂进去的，恰好绕开了 M3 这个不可复现环节。

3.6 导出加速：Open3D 写 OBJ 的问题

复用命中后单件只剩 6.2 s，其中 **6.05 s 全在导出。**拆开看：写 OBJ 5.34 s、写 GLB 0.97 s、写顶点色 PLY 0.08 s。原因在 Open3D 的 OBJ 写出器：它把 UV 按**逐角**重复写出（73751 面 → **221253 条 vt**，是顶点数的 4.3 倍），并且逐行格式化、逐值转字符串。

改为手写：v / vt / vn 各一块用 numpy 批量格式化，f 用 v/vt/vn 同号（xatlas 已按 vmap 拆开接缝，每顶点只有一个 UV，所以逐顶点 vt 与逐角 vt 语义等价，且体积更小），MTL 与 _diffuse.png 直接落地——顺带消掉了 Open3D 先写 _0.png 再由 _normalize_obj_texture 改名的那一步。

项	Open3D 写出	手写（本版）	等价性核对
耗时（37k 顶点 / 74k 面）	5.34 s	0.21 s	25 倍
vt 条数	221253（逐角）	51105（逐顶点）	回读逐角 UV 与源一致（max diff 5.1e-7）
V 方向	原值	原值（**不翻转**）	逐值对照 uv[i] 与 vt 行
几何精度	%.6g	%.6f	排序后 max diff 9.5e-7（本版更准）
旁挂文件	_0.png → 改名 _diffuse.png	直接写 _diffuse.png	不再产生 _0.png

> **退路**：拿不到逐顶点 UV（例如外部传入逐角 UV 且与顶点数不匹配）时，_write_obj_textured() 返回 False，导出自动回退 Open3D 路径并写 WARNING——不静默降级，也不会写出贴图错位的 obj。

四、实测数据

测试数据取真机项目 D:\h3d_relic_test 的 relic_001（30 张 4496×3000 照片、37k 顶点网格、1024² 图集），复制成 5 件同几何/同位姿/同照片（照片用硬链接，不额外占盘），在**同一台笔记本**上分别关闭与开启复用各跑一遍。

4.1 关闭复用（基线）

件	总耗时 s	UV s	采样+融合 s	导出 s	缓存状态
relic_b1	24.4	10.9	13.1	0.4	miss
relic_b2	22.5	9.9	12.2	0.4	miss
relic_b3	22.7	10.1	12.3	0.4	miss
relic_b4	23.0	9.9	12.8	0.4	miss
relic_b5	24.4	10.9	13.1	0.4	miss
合计	**约 115**	—	—	—	5 件全算

4.2 开启复用

件	总耗时 s	UV s	采样+融合 s	导出 s	缓存状态	来源
relic_b1	26.4	11.0	14.3	1.2	miss（建缓存）	—
relic_b2	**0.5**	0.01	0.14	0.39	hit	relic_b1
relic_b3	**0.6**	0.01	0.14	0.40	hit	relic_b1
relic_b4	**0.6**	0.01	0.15	0.39	hit	relic_b1
relic_b5	**0.5**	0.01	0.14	0.39	hit	relic_b1
合计	**约 29**	—	—	—	4 件命中	省 74.8%

第 1 件比基线略慢（26.4 s vs 24.4 s），因为要多算三个指纹并写缓存（照片内容 SHA-1 约 0.3 s + 缓存写盘 <2 MB），这是**一次性成本**；从第 2 件起 **0.5~0.6 s**，单件省 **97.7%**。

4.3 画质不变的证据

件	关闭复用 贴图 SHA-1	开启复用 贴图 SHA-1	结论
relic_b1	224e0f8fe9b3a092a256be1450115261fb16e256	224e0f8fe9b3a092a256be1450115261fb16e256	一致
relic_b2	224e0f8f…（同）	224e0f8f…（同）	一致
relic_b3	224e0f8f…（同）	224e0f8f…（同）	一致
relic_b4	224e0f8f…（同）	224e0f8f…（同）	一致
relic_b5	224e0f8f…（同）	224e0f8f…（同）	一致

5 件的 _L1.0_diffuse.png 在两种模式下都是同一个 SHA-1，也就是说**复用产物与重新计算的结果逐字节相同。**这条证据比「看上去一样」更有力：它同时排除了「颜色整体偏移」「掩膜不一致」「V 翻转」这类只靠目视容易漏掉的差异。

4.4 与上一轮真机数据的关系

上一轮真机 5 件的 M4 是 83~85 s/件（UV 33 s + 采样 50 s），比本批复现数据的 22.5~24.4 s 高，差异来自网格规模与图集分辨率。两者**结构一致**：UV + 采样占 96%~98%，导出占 2%~4%。因此按同一比例外推：

场景	关闭复用（上一版）	开启复用（本版）	说明
真机 5 件（83~85 s/件）	约 7 min	首件约 85 s + 后 4 件各约 2 s ≈ 1.6 min	按实测比例推算，待下次真机复测确认
本批复现 5 件（22.5~24.4 s/件）	115 s	29 s	实测
单件增量（第 2 件起）	22.5~24.4 s	0.5~0.6 s	实测

4.5 打包版端到端（2 件同源输入）

把 1.1.6 的 exe 拉到 `D:////_h3d_pkg_m4b`（两件输入完全相同：同 raw、同 30 张照片、同 capture_meta），执行 `H3D.exe --batch config.yaml`：total=2 done=2 failed=0，第 2 件日志两行——

```
[INFO] [h3d.photo_texture] UV 图集复用：demo_b（几何指纹 7f2bf76beb54，来自 demo_a，跳过 xatlas 展开）
[INFO] [h3d.photo_texture] 纹理图集复用：demo_b（来自 demo_a，跳过逐纹素采样与融合）
```

修复前同一项目跑出来是 **两个不同的几何指纹、两个不同的纹理键**（缓存目录里各有两套），即命中率为 0。这个对比是本批最有价值的一次验证：它证明「两级缓存实现正确」与「缓存能在真实批处理里命中」是两件不同的事。（该测试项目的位姿与几何来自不同坐标系，贴图填充率为 0，属测试夹具问题，与复用逻辑无关；像素一致性由 4.1~4.3 的真机数据基准负责。）

五、测试与验收

全量 **705 项通过**（1.1.5 为 682 项，本批 +23）。新增测试不满足于「不报错」，而是锁定行为本质：

测试文件	新增	锁定的行为
tests/test_texture/test_atlas_cache.py	20	指纹精度（1e-10 不敏感、1e-3 敏感）；位姿顺序无关但改 t 必变；UV 往返逐位一致；损坏文件留档且判未命中；纹理缺一件即未命中；关闭开关则完全不写盘；体积上限生效；**端到端第 2 件命中且贴图逐像素一致**；同 root 换照片 → UV 命中/纹理未命中（partial）；metrics 可 JSON 序列化
tests/test_mesh/test_reconstruct.py	1	默认线程数下泊松两次运行顶点/面表哈希完全相同（多线程下该断言会失败）
tests/test_texture_export.py	2	手写 OBJ 为逐顶点 vt、mtllib/usemtl 正确、回读几何与逐角 UV 与源一致（V 不翻转）；拿不到逐顶点 UV 时返回 False 让调用方回退

端到端用例里那条 `np.array_equal(a, b)` 是最关键的一条：它比较的是两件产出的贴图像素本身。如果哪天指纹漏了某个参数（比如忘了把 `top_k` 放进 key），这条断言会立刻失败——而只断言「运行成功」的测试不会。

六、风险与未做

项	现状	影响 / 处置
缓存按项目根隔离	缓存位于 <根>/99_runs/atlas_cache	跨项目不共享（避免串味）。若真有多项目同批照片，各自首件各建一次缓存
照片改名会换指纹	指纹含文件名	同内容不同名 → 纹理级未命中（UV 级仍命中）。这是**故意的保守选择**：文件名进指纹与 S1 复用口径一致
法线抖动	法线参与纹理级键	若某件的 M1 法线朝向投票结果不同 → 纹理级未命中、UV 级仍命中（退化为只省 33 s）
缓存体积	24 套 / 约 3 MB 每套（1024^2）	doctor 的 atlas_cache 检查项会提示；可随时删除，只损失速度
泊松单线程代价	37k 点 depth=8：0.84 s → 1.51 s	大点云（>50 万）代价更高；可用 mesh.poisson_threads 换速度，但那样图集复用与可复现性同时失效
M4 剩余成本	命中后 0.5~0.6 s/件，其中 0.39 s 是导出	已无大幅优化空间；若后续把 GLB 也改成手写可再省约 0.2 s
真机 5 件复测	未做	需用 v1.1.6 打包版在真实项目上跑一遍，核对 atlas_reuse 与整批时长

七、下一步

① 用 v1.1.6 打包版在真实 5 件项目上复测，确认 M4 从 83~85 s/件 降到秒级、整批从约 7 min 降到约 1.6 min，并核对报表里 `atlas_reuse` 的来源件号。② 后续可把纹理级复用的判据从「法线指纹」放宽为「法线朝向一致性判据」（只比较朝向投票结果，不比较逐顶点数值），以避免法线微差导致的无谓未命中。③ GLB 写出同样交给手写实现（当前 trimesh 约 0.97 s/件）。④ 若将来引入多机位混拍（多相机内参），位姿指纹已按 `cam_id` 分桶，可直接支持。